Component Model Update

Bytecode Alliance Plumber's Summit - October 2026

Luke Wagner

Last time

- Presented a roadmap at the last (February) Plumber's Summit (slides)
- Pretty much following that roadmap since then…

Outline

- Done
- In progress
- Next
- Remaining for 1.0

Done

Released Preview 3 (0.3.0) \o/

- Containing:
  - async functions, streams, futures, waitable sets, cancellation, backpressure, readiness, an async call stack, upgraded WASI interfaces, >5 languages bindgen, >2 hosts, …
  - https://bytecodealliance.org/articles/WASI-0.3
- Phew, that was a lot of work, many thanks to so many people!!
- “Releasing” is just the beginning: unblocks toolchain upstreaming
  - E.g.: --target=wasm32-wasip3 as Tier 2 target in Rust
  - More from Alex later today

Released 0.3.1

- (import “redis” (**implements** “wasi:keyvalue/store”) (instance …))
  - Multiple imports of the same WIT interface (avoid “uniqueness” limitation)
    - Other examples: HTTP backends, secrets
  - “redis” is the import’s *name*, providing uniqueness (for linking)
  - (implements …) is an *attribute*: metadata, not part of name, not part of the type
- (import "my-pkg” (**external-id** “https://unpkg.com/…”) (instance …))
  - When you need an arbitrary Unicode string to *identify* something in the outside world
    - Other examples: OCI URL, any non-kebab-case platform-defined names (e.g., web APIs)
  - Again, “my-pkg” is the import’s *name*, providing uniqueness + kebab-case (for bindgen)
  - (external-id …) is also an *attribute*, composes with (implements …)
- **map**<K,V> (thanks Yordis Prieto!)
  - Idiomatic bindings to: maps, dictionaries, (incoming) objects
  - Not quite enabled in Wasmtime yet

Polished cooperative threads for 0.3.{3,4}

- “Cooperative” = “not preemptive”, switch only at cooperative yield points
  - Preemptive threads depend on **shared** functions in Core WebAssembly, will take much longer
- Use cases:
  - Pthreads (externally scheduled; as-if running on single core)
    - Unblocks Python from switching to wasip3 and the Component Model (PEP 816)
  - Stackful coroutines/goroutines (internally scheduled by guest languages’ own scheduler)
  - Instead of today using Asyncify / CPS transform ⇒ really just a (size+speed) optimization
- Semantically defined in terms of stack-switching, but not dependent on it:
  - … which is good b/c stack-switching design may be changing again (stack-10-19)
  - Everything defined as “ABI built-ins” imported by core wasm as functions, e.g.:
    - (**import** “thread.new-indirect” (**param** $funcIndex **i32**) (**param** $voidPtr **i32**) (**result i32**))
    - (**import** “thread.suspend-then-resume” (**param** $threadIndex **i32**) (**result i32**))
- Implementation:
  - Initial sketch/implementation work last year (thanks Sy Brand!)
  - Bunches more testing, polish, spec tweaks earlier this year
  - More from Alex on toolchain progress and plans later today

Added lots of tests and aligned spec/impl

- Tons of interesting tests now
  - ~26 KLOC of WAST under component-model/test/
  - Unit tests, but also some bigger integration tests (e.g. test/async/big-interleaving-test.wast)
  - Found and fixed lots of impl and spec bugs/divergences
- AI is a pretty great tool for writing WASTs
  - LLMs can already one-shot complex Preview 3 component WAST from pre-training (!!)
    - Consequence of: clear set of regular rules + lots of examples that follow the rules
  - Can emit WAST for new unimplemented features purely based on spec
    - Really hard to do by hand before there is a working impl to test against
    - Avoids biasing test results towards “what happened in Wasmtime”
    - Provides early “implementation” feedback before real implementation work

Removed recursion restrictions

- Recursion disallowed since Preview 2, so Preview 3 tried to uphold
  - Why?
    - Less hard cases for compiler/bindgen to worry about
    - Usually not expected by guest programs (fun example: “The DAO hack”)
  - However: hard to precisely distinguish “good” (“sibling”) from “bad” (“recursive”) reentrance
    - Preview 3 requires allowing lots of reentrance
    - Ultimately, trying to rule out only the “recursive” kind seemed like a bit of a fig leaf
- So, removed the recursion restriction (remove the trap)
  - Wrote lots of WASTs, fixed lots of interesting corner cases
- Cool: “donut-wrapping” composition is now possible (in WAT / runtime)!
  - Avoid the extra memory of “hamburger wrapping” when virtualizing
  - Still missing: language bindings to emit donut WAT in each language (JS, Rust, …)
  - (BA member) Mimic is already emitting donut-wrapping from their internal language!

Added memory64 spec rules

- Added spec rules in CM/#624 (thanks Adam Bratschi-Kaye!)
- Implementation not started (it'll be a bit hefty)
- Good to see how it'd work, ensure it integrates with other features in design

In progress

Firefox Web Preview

- You may have seen Ryan Hunt's Mozilla Hacks Post:
  - Why is WebAssembly a second-class language on the Web?
- Next step: "Web Preview" in Firefox Nightly:
  - `<script src="./component.wasm" type="module">` (via ESM-integration)
  - `(import "wasm:js/global" (instance (export "node" (type ...) ...))` (via JS built-in)
  - Now I can create a whole web app with just HTML + WebAssembly!!
    - with >= performance than JS or JS+wasm
  - `import { my-export } from "./component.wasm"` (via ESM-integration)
  - `WebAssembly.instantiateStreaming(fetch('component.wasm'), ...)` (via JS API)
  - Spec PR: CM/#686 (thanks again Ryan!)
- Supporting efforts on the BA and CM side:
  - Add `--target=wasm32-webp2` to wasi-sdk, wasi-libc, Rust, … ⇒ more from Alex later
  - Temporary workaround for current lack of callbacks using resource types + *recursion*: CM/#743
  - Still needed in the CM: resource subtyping, getters/setters, overloads, unions (next…)
- Links:
  - Mozilla: Intent-to-Prototype, standards-positions: positive, bugzilla meta-bug
  - Bytecode Alliance: #web-preview Zulip channel

Getters and setters

- Recently added to spec in CM/#701 (thanks Ben Visness!)
  - In WIT:
    - **resource** blob {
      position: **get**() → **u64**;
      position: **set**(value: **u64**);
      }
  - Turns into WAT:
    - (**import** “[method][get]blob.position” (**func** (**param** “self” (**borrow** $blob)) (**result u64**)))
    - (**import** “[method][set]blob.position” (**func** (**param** “self” (**borrow** $blob)) (**param** “v” **u64**)))
  - Also have “[static][get/set]...” and plain “[get/set]...”
- Useful for Web Preview web API bindings to avoid JS glue code
- Implementation in progress in Wasmtime (-W component-model-accessors=y)
  - To be released in some future 0.3.*

Stream and future forwarding

- New ABI builtins:
  - {stream,future}.forward $T : [readable-endT:**i32** writable-endT:**i32**] → []
  - Removes both ends from caller's table, allows eager instance teardown
  - Spec PR: CM/#717, Wasmtime implementation in progress (thanks Roman Volosatovs!)
- Postponing more-complex "splice" operation:
  - For passing "some" elements (not "all the rest")
  - Original idea: stream.splice $T : [rstreamT:**i32** wstreamT:**i32** n:**i32**] → [status:**i32**]
  - New idea by way of Roman:
    - stream.split $T : [rstreamT:**i32** n:**i32**] → [head-rstream:**i32** rest-rstreamT:**i32**]
    - stream.join $T : [head-rstreamT:**i32** rest-rstreamT:**i32**] → [rstreamT:**i32**]
  - Complementary/symmetric with sketches for future "**stream<T then U>**"
    - stream-future.split $T $U : [rstreamTU:**i32**] → [rstreamT:**i32** rfutureU:**i32**]
    - stream-future.join $T $U : [rstreamT:**i32** rfutureU:**i32**] → [rstreamTU:**i32**]
  - Waiting until someone needs to use it…

Switching to “canonical versions”

- Spec’d a while ago (CM/#536, thanks Lann Martin!)
- Idea:
  - Instead of: (import “wasi:http/handler@0.3.1” (instance …))
    - … requiring custom semver logic for forward+backward semver compat everywhere
  - have: (import “wasi:http/handler@0.3” (version-suffix “.1”) (instance …))
    - … so that simple name-string equality does the right thing
    - (version-suffix …) is an attribute used for error messages
- Implementation almost done (thanks Yan Chen!)
  - Multi-stage roll out: accept in Wasmtime → emit from wasm-tools → deprecate non-canonical

Next

Resource subtyping

- Old good idea, now motivated by Web Preview:
  - **resource** event-target { add-event-listener: **func**(…); … }
    resource node **extends** event-target { … }
  - … turns into …
  - (**import** “event-target” (**type** $EventTarget (**sub resource**)))
    (**import** “node” (**type** $Node (**sub** $EventTarget)))
- At runtime: **i32** handle type checks accept event-target *or* node
- No virtual dispatch, no general-purpose runtime “downcast”
  - But web APIs need downcasts (consider: `get-element-by-id: **func**(id: **string**) → element`)
  - So as a short-term solution we can write in WIT today:
    - to-node: **func**(super: **borrow**<event-target>) → **option**<**own**<node>>; // for static langs.
      // and so on for each other target type (element, HTML-element, …)
    - specialize: **func**(super: **borrow**<event-target>) → most-specific-type; // for dynamic langs.
      variant most-specific-type {
      node(**own**<node>),
      // and so on for for each other target type (element, HTML-element, …)

Unions

- Motivated by Web Preview and web APIs in CM/#724
- Early CM had unions, which were removed
- This time around:
  - Union case payload types must be “distinguishable” (spec-defined predicate)
    - Allow static languages to use static overload resolution
    - Allow dynamic languages to use runtime type tests (like web APIs do today in JS)
  - Union cases have names (just like variant cases) as fallback:

```
union node-or-string {
  node(own<node>),
  text(string)
}
```

Overloading

- Motivated by Web Preview and web APIs in CM/#742
- Not sure what the WIT syntax looks like…
- At WAT level, give overloads distinct names, e.g.:
  - (**import** “[method][overload]canvas.draw-image#pt” (**func** …))
  - (**import** “[method][overload]canvas.draw-image#rect” (**func** …))
  - (**import** “[method][overload]canvas.draw-image#src-to-dst” (**func** …))
- Language bindings overload if they can, fall back to explicit name suffix if not
- Extend the (union) “distinguishable” predicate to function types

Optional imports

- Help us resolve these competing constraints:
  - Want our producer tools to be able to generate components that target minimal/custom worlds
    - For “little plugins”, UDFs, functional pipelines, AI sandboxes, POLA, …
  - Want to upstream WASI support everywhere so things Just Work (esp. the standard library)
    - Lots of WASI imports get pulled in unconditionally, have to stub out / virtualize
  - Statically gating via `#[cfg(feature = “wasi:filesystem/types”)]` is kind-of a pain
    - Need to make N different builds of standard library, plumb more info more places
- With optional imports
  - Import WASI (and/or Web) support everywhere… optionally
    - **import optional** wasi:filesystem/types
  - Get a feature-test bool from the Canonical ABI “for free”:
    - (**import** “wasi:filesystem/types.is-present” (**global i32**))
    - (**import** “wasi:filesystem/types.descriptor.open-at” (**func** …)) ;; trap if not present

Next - ABI v.2 (“lazy lowering”)

- Apply lessons learned since Previews 2 and 3
  - Include feedback from the new CM SG
- Incremental transition
  - Add opt-in ABI option → switch producer tools → default/only option in 1.0
- Lazy lowering benefits:
  - Kill cabi-realloc, zero-cost forwarding, graceful OOM, recursive values in the future (e.g. JSON), same-instance-stream, per-value ABI options, …
- Not just lazy lowering:
  - Multi-value return
  - Error-context in result
  - “Stackful” async ABI
  - Newer idea: for dynamically-sized return values not-in-lists (e.g. `**func**() -> **result**<**string**>`)
    - Pass caller-supplied buffers in outparam area with lazy-lowering as fallback
    - Near-optimal ABI for libc: read: **func**(fd: **borrow**<fd>, n: **u32**) → **result**<**list**<**u8**>>

Still to do before 1.0 (probably)

- Guest C ABI
  - “Just give me a .h to call/implement” for producer toolchains
- wasm-gc ABI
  - For Kotlin, Scala, Dart, Java, OCaml, …
- Function subtyping (“API evolution”)
  - Add optional function parameters, optional record fields, variant/enum cases
- Custom attributes
  - For all the ∞ custom use cases we haven’t addressed yet (see sketch in CM/#795)
- Callbacks
  - Maybe along with `weak<R>` handles to break ownership cycles…
- Better downcast support for web APIs (?)
  - More modular, more efficient, but still virtualizable

Ways to engage

- Try it out
  - https://component-model.bytecodealliance.org/
- New WAT-level walkthrough
  - https://github.com/WebAssembly/component-model/pull/745
- (Bytecode Alliance) SIG-Component-Model:
  - https://github.com/bytecodealliance/meetings/tree/main/SIG-Component-Model
- (W3C) Component Model Subgroup
  - https://github.com/webassembly/meetings/tree/main/component-model/
- Web Preview:
  - https://bytecodealliance.zulipchat.com/#narrow/channel/634524-web-preview